

Essential Python Tools for Data Cleaning

When you begin cleaning data in Python, two essential libraries stand out: **pandas** and **numpy**. These libraries are widely used because they make it simple and efficient to load, inspect, and transform data. **pandas** is designed for working with structured data, such as tables and spreadsheets, using its powerful `DataFrame` and `Series` objects. With pandas, you can easily filter, sort, aggregate, and reshape your data. **numpy** focuses on numerical operations and provides fast, flexible tools for working with arrays of numbers. Combining pandas and numpy gives you a strong foundation for handling missing values, correcting data types, and performing calculations that are common in real-world data cleaning tasks.

```
import pandas as pd
import numpy as np

# Create a simple pandas DataFrame
data = {
    "name": ["Alice", "Bob", "Charlie", "David", np.nan],
    "age": [25, 30, np.nan, 22, 28],
    "score": [88.5, 92.0, 85.0, np.nan, 90.0]
}
df = pd.DataFrame(data)

# Inspect the DataFrame
print("DataFrame head:")
print(df.head())

# Check for missing values
print("\nMissing values in each column:")
print(df.isnull().sum())

# Fill missing ages with the mean age using numpy
mean_age = np.nanmean(df["age"])
df["age"] = df["age"].fillna(mean_age)
print("\nDataFrame after filling missing ages with the mean:")
print(df)

# Convert all names to lowercase using pandas string methods
df["name"] = df["name"].str.lower()
print("\nDataFrame after standardizing names to lowercase:")
print(df)
```



You can see how **pandas** makes it easy to inspect your data, check for missing values, and apply transformations. **numpy** is often used alongside pandas to perform numerical calculations, such as finding the mean of a column while ignoring missing values. By combining these libraries, you can quickly prepare your data for further analysis or modeling.